

Model Checking MCP-Style Tool-Using Agents: Accuracy of LLM-Written Models and Reproducibility of Counterexamples

CS 3892, Fall 2026 Project proposal Group B

Teo Kitanovski, Connor Brugger, Xiaodi Shao

<https://github.com/ConnorBrug/mcp-agent-verification>

Acronyms used below: MCP is the Model Context Protocol, the standard way an LLM (large language model) agent connects to its tools. LTL is linear temporal logic. SMV is the input language of the NuSMV and nuXmv model checkers. BDD is binary decision diagram and BMC is bounded model checking. FM4AI means formal methods for AI, and AI4FM means AI for formal methods.

1. Topic

Menu topic 8, formal models of agentic AI. We model an LLM agent that calls tools through MCP as a finite transition system in SMV, write safety properties in LTL, and check them with nuXmv. The project asks three questions.

- **RQ1.** How accurately can an LLM translate real MCP tool schemas into a formal transition-system model? We compare each generated model with a hand-written reference on whether it parses, whether it gets the same verdict for each property, and whether it admits the reference’s counterexamples.
- **RQ2.** What security violations can we discover by model checking the LLM-generated models? We count a violation as confirmed only when the reference model also admits its counterexample. Anything else is a translation error.
- **RQ3.** How many of the attack traces from the model checker reproduce against a sandboxed agent that uses the corresponding real tools?

The work covers 5 scenarios. Each has a hand-written reference model in two variants, unguarded and guarded. The LLM writes 10 models per scenario, 50 in total. Five come from the tool schemas alone and are compared with the unguarded reference. The other five also get an English description of the policy and are compared with the guarded reference. Every model is checked against the same 3 properties.

2. The FM half

The method is LTL model checking of finite-state systems, and one of the properties uses a past-time operator. The tool is nuXmv 2.2.0 with its BDD engine in batch mode. NuSMV 2.7 is our fallback, and nuXmv’s BMC engine is available if a model gets too large for BDDs.

All models, reference and generated, declare the same interface, so one property file checks every model. The interface is `action ∈ {idle, read_trusted, read_untrusted, send_external, confirm, delete, request_grant}` and four booleans: `tainted`, `confirmed`, `privileged`, and `grant`. The LLM’s next tool call is a nondeterministic input. A second input, controlled by the attacker, decides whether an untrusted read contains a prompt injection. If it does, `tainted` becomes true and stays true. Each scenario maps its tools onto the abstract actions. In AgentDojo, for example, `send_money` and `send_email` map to `send_external`, `delete_file` maps to `delete`, and `get_webpage` and `read_inbox` map to `read_untrusted`. The guarded variant puts a policy layer in front of execution. It drops an external send while the agent is tainted, drops a delete unless the previous step was a confirm, and grants privilege only when `grant` is true.

The properties, where **G** means always, **X** the next step, and **Y** the previous step:

- P1, no exfiltration after injection **G**(`tainted` → **G**(`action` ≠ `send_external`))
P2, delete needs confirmation **G**(`action` = `delete` → **Y**(`action` = `confirm`))
P3, no privilege without a grant **G**((¬`privileged` ∧ **X**`privileged`) → `grant`)

On unguarded models we expect nuXmv to return *false* for all three properties, each with a lasso-shaped counterexample. On guarded models we expect *true*. A generated model should get the same verdict as its reference, and a mismatch counts against it in RQ1.

We have already run nuXmv on toy versions of both reference variants, and `make check` gives the expected verdicts:

```
== agent_unguarded
COUNTEREXAMPLE G (action = delete -> Y action = confirm)
COUNTEREXAMPLE G (tainted -> G action != send_external)
COUNTEREXAMPLE G ((!privileged & X privileged) -> grant)
== agent_guarded
HOLDS          G (tainted -> G action != send_external)
HOLDS          G (action = delete -> Y action = confirm)
HOLDS          G ((!privileged & X privileged) -> grant)
```

In the P1 counterexample, the agent reads untrusted input that carries an injection and sends data out on the next step. Our script `eval/parse_trace.py` converts traces like this one to JSON for replay.

3. The AI half

The AI is mainly the subject (FM4AI), since we model and check the behavior of an LLM tool-using agent, but we also use an LLM as an instrument (AI4FM) to write the SMV models, and RQ1 evaluates how well it does.

4. Data or benchmark

- AgentDojo (Debenedetti et al., NeurIPS 2024 Datasets and Benchmarks track, MIT license, <https://github.com/ethz-spylab/agentdojo>). It provides stateful simulated tool environments along with prompt-injection attacks. We take the tool schemas for 4 scenarios from it and use it as the sandbox for RQ3. Its tool definitions export directly to MCP’s tool format (name, description, JSON Schema `inputSchema`). We checked access on 27 Sep 2026: `pip install agentdojo` installed v0.1.35, and `get_suites("v1.2")` loaded the workspace (24 tools), travel (28), banking (11), and slack (11) suites, for 74 tools, 97 user tasks, and 35 injection tasks in total.
- The MCP reference servers (<https://github.com/modelcontextprotocol/servers>). Scenario 5 is an agent connected to the fetch (1 tool), filesystem (14), and git (12) servers. In this scenario `fetch` is an untrusted read, and it can also leak data through the URL it requests. We checked access by cloning the repository at commit `f46d957` on 27 Sep 2026. Each server’s tool definitions are in its source.
- We write the 5 reference models and the property file ourselves. nuXmv produces the counterexamples from them.
- Neither dataset needs credentialing. Model generation and agent-driven replay use Vanderbilt’s Amplify API, which gives university accounts access to GPT, Claude, and Mistral models. Amplify returns plain text without native tool calling, so agent-driven replay uses AgentDojo’s prompt-based tool-calling agent (`PromptingLLM`).

5. Success criterion

We set these thresholds before generating any models. The metric formulas are in `docs/metrics.md` in the repository.

- **RQ1 (primary).** Across the 50 generated models, strict verdict agreement with the reference (VA) is at least 0.8 and the false-safe rate (FS) is at most 0.1. A false-safe result is a generated model reporting *true* for a property its reference violates. We also report parse rate and trace admissibility.
- **RQ2.** Checking the generated models finds a counterexample to each of P1, P2, and P3 in at least 3 of the 5 scenarios, and the reference model admits at least 70% of all the counterexamples found.
- **RQ3.** In agent-driven replay, counterexample traces cause a violation more often than random action sequences of the same length (one-sided test, $p < 0.05$).

If RQ1 comes out negative, with VA below 0.8 or FS above 0.1, LLM-written models need human review before anyone trusts their safety verdicts. We would then report which properties and which tool mappings the LLM got wrong. If fewer than 70% of the RQ2 violations are confirmed, checking LLM-written models mostly finds translation errors and not attacks. If counterexample traces reproduce no more often than random ones, our abstraction is too coarse to predict how real agents fail, and we report how many traces were spurious. Each of these outcomes is still worth reporting. People are starting to ask LLMs to write formal models, and a measured failure shows where that breaks and which kinds of tools or properties cause it.

6. Responsibility breakdown

Teo Kitanovski	Reference models and properties for all 5 scenarios, nuXmv runs (RQ2), and the metric scripts in <code>eval/</code> .
Connor Brugger	The generation pipeline in <code>pipeline/</code> : prompts, the 50 generation runs, the parse-and-repair loop, and run logs (RQ1).
Xiaodi Shao	The replay harness in <code>sandbox/</code> on AgentDojo, the maps from abstract actions to concrete tool calls, and both kinds of replay (RQ3).
All	Interface spec, report, talk, and the AI-use log in <code>docs/AI_USE.md</code> .

We plan to have reference models for all 5 scenarios and a working generation pipeline by 15 Oct. By the 12 Nov milestone we will have checked all 50 generated models, computed the RQ1 and RQ2 metrics, and have scripted replay running. The milestone demo is `make check` followed by a replay of one counterexample. Agent-driven replay and the RQ3 numbers come last, for the final report.

7. Repository URL

<https://github.com/ConnorBrug/mcp-agent-verification>. The repository has a README with the research questions, install steps, and a run guide. It also holds the interface spec, the metric definitions, the two toy reference models, the property file, a Makefile, and the trace parser. Running `make check` reproduces the toy result in

Section 2.

8. Risks and fallback

The most likely failure is in RQ3, where abstract traces have to become concrete tool calls. A counterexample step such as `send_external` does not say which tool to call or with what arguments. An LLM agent may also ignore an injected instruction for reasons unrelated to our model, so reproduction rates could come out near zero without telling us anything about the abstraction. If that happens, we split RQ3 in two. In scripted replay we issue the concrete calls ourselves from a per-scenario action map, which tests whether the environment allows the violation at all. In agent-driven replay we plant only the injection and watch what the agent does. If agent-driven replay turns out too expensive or too noisy, we run scripted replay on every trace and agent-driven replay on a fixed subset.

The second risk is that generated models fail to parse or rename interface variables, which would leave RQ1 with nothing to compare. The fallback is to give the LLM the interface declarations as a fixed template and let it repair its model up to 3 times from nuXmv’s error messages. We would report parse rate before and after repair.

Amplify usage limits could also cap how many models we generate. We fix generation at 10 runs per scenario and commit every output, so no run has to be repeated.